Faculty of Engineering and Technology
Department of Electrical and Computer Engineering

ASSIGNMENT 1

Distributed Proof-of-Work Nonce Search with REST and gRPC

Technical report · Parts 1–4 and bonus dashboard

PREPARED BY	Yahya Sarhan
STUDENT ID	1221858
INSTRUCTOR	Dr. Abdalkarim Awad
COURSE	ENCS5324 — Advanced Distributed Systems
SUBMISSION DATE	21 November 2026

Abstract

This report presents the design, implementation and evaluation of two Go systems built for ENCS5324 Assignment 1. **Part 1** answers the conceptual questions on synchronous and asynchronous RPC, the four gRPC communication patterns, REST versus gRPC, and long-running REST operations with Server-Sent Events (SSE). **Part 2** implements a gRPC Live Log Viewer with one unary and one server-streaming method, verified through manual runs, client-side cancellation and automated tests.

Part 3 implements a distributed SHA-256 proof-of-work nonce search. A coordinator exposes a REST/SSE API to clients and distributes nonce ranges to independently running gRPC workers, with early termination, client-initiated cancellation, heartbeats and automatic range reassignment. **Part 4** reports all required experiments; every nonce reported in this document was re-hashed independently and is valid. In the scalability test ($d = 5$), the median job time fell from 22.05 ms with one worker to 6.46 ms with eight (3.41×). The analysis shows that this gain comes mainly from parallel ranges reaching an earlier solution (156,057 hashes with one worker versus 28,024 with eight), while fixed coordination overhead dominates jobs this short.

Contents

1. Concepts — Written Answers 4
1.1 Synchronous vs. Asynchronous RPC 4
1.2 The Four gRPC Communication Patterns 4
1.3 REST vs. gRPC 6
1.4 Long-Running REST Operations and SSE 6

2. A Streaming RPC Service in Go 8
2.1 Project Overview 8
2.2 Protocol Definition 8
2.3 Implementation Summary 8
2.4 Execution and Test Results 9

3. Distributed Proof-of-Work Nonce Search 11
3.1 Objective and Proof-of-Work Rule 11
3.2 System Architecture 11
3.3 REST API and Job Lifecycle 12
3.4 gRPC Services and Worker Registration 12
3.5 Range Scheduling and Concurrent Jobs 13
3.6 Early Termination, Cancellation and Fault Tolerance 13
3.7 Configuration and Running the System 14
3.8 Optional Browser Dashboard (Bonus) 16

4. Testing and Experiments 17
4.1 REST Workflow and SSE 17
4.2 Difficulty Experiment (d = 4, 5, 6) 17
4.3 Scalability: 1, 2, 4 and 8 Workers 18
4.4 Concurrent Jobs 20
4.5 Client-Initiated Cancellation 21
4.6 Fault Tolerance and Range Reassignment 22
4.7 REST Error Handling 22
4.8 Automated Tests, Bonus and Limitations 23

5. Conclusion 24

Appendix A — Source Code, Build and Reproduction 25

Appendix B — Independent Hash Verification 26

Appendix C — References 26

Appendix D — Benchmark Screenshots 27

1. Concepts — Written Answers

1.1 Synchronous vs. Asynchronous RPC

A Remote Procedure Call (RPC) lets a client invoke an operation that runs on a remote server as if it were a local function. The two styles differ in what the caller does while the remote work is in progress.

- **Synchronous RPC:** the client sends the request and **blocks** until the response arrives. The code is simple and reads like a normal function call, but the calling thread can do nothing else in the meantime.
- **Asynchronous RPC:** the client sends the request and **continues immediately**. The result is delivered later through a future, callback, channel or completion event, so the client can do other work or issue many calls in parallel.

When synchronous is better: checking login credentials. The next step depends entirely on the answer and the check takes only milliseconds, so blocking is simple and costs almost nothing.

When asynchronous is better: a long-running task such as video processing, or a coordinator that must contact several workers at once. In Part 3, the coordinator starts a Search call on every worker without waiting for the previous one to finish, and handles each result as it arrives.

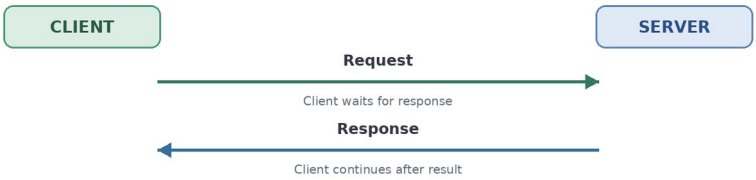


Figure 1. Synchronous RPC: the client waits for the response.



Figure 2. Asynchronous RPC: the client continues before the result arrives.

1.2 The Four gRPC Communication Patterns

gRPC methods differ in whether each side sends a single message or a stream of messages within one call. This gives four method types:

Table 1. The four gRPC method types.

Method type	Client sends	Server sends	Realistic example
Unary	One request	One response	Validate login credentials; fetch an account balance
Server streaming	One request	Many messages	Live application logs; a stock-price feed after one subscription
Client streaming	Many messages	One response	Upload a large file in chunks, then receive one confirmation
Bidirectional streaming	Many messages	Many messages	Real-time chat or a multiplayer game syncing state

- **Unary:** behaves like an ordinary function call: one request in, one response out. In Part 3, `RegisterWorker` and `Heartbeat` are unary.
- **Server streaming:** the client asks once and the server sends a sequence of messages over time until it closes the stream. Part 2's `WatchLogs` and Part 3's `Search` both use this pattern.
- **Client streaming:** the client sends a sequence of messages and the server replies once at the end, for example after all file chunks have arrived.
- **Bidirectional streaming:** both sides read and write independent streams over the same call at the same time.

PROTOCOL BUFFERS SYNTAX FOR EACH PATTERN

```
rpc Login(LoginRequest)          returns (LoginResponse);     // unary
rpc WatchLogs(LogRequest)        returns (stream LogMessage); // server stream
rpc UploadFile(stream FileChunk) returns (UploadResponse);    // client stream
rpc Chat(stream ChatMessage)     returns (stream ChatMessage); // bidirectional
```

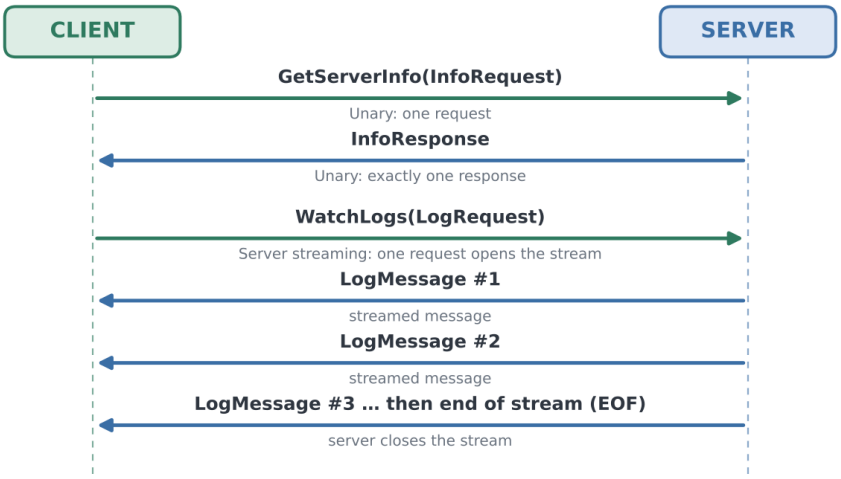


Figure 3. Message flow for a unary call followed by a server-streaming call (Part 2 Log Viewer).

1.3 REST vs. gRPC

Similarities

- **Client–server request/response model:** both let a client invoke functionality on a remote service and receive a result.
- **Built on HTTP:** REST runs over HTTP/1.1 or HTTP/2; gRPC runs over HTTP/2. Both use headers or metadata and status codes.
- **Language- and platform-independent:** a Go server can serve clients written in Python, Java, JavaScript or C#.
- **Production features:** both support TLS encryption, authentication tokens, timeouts and load balancing.

Differences

Table 2. Main differences between REST and gRPC.

Aspect	REST	gRPC
API style	Resource-oriented: nouns plus HTTP verbs (`GET /jobs/5`)	Action-oriented: call remote methods (`Search()`, `Heartbeat()`)
Data format	Usually JSON: text, human-readable, larger	Protocol Buffers: binary, compact, faster to parse
Contract	Optional (e.g. OpenAPI); loosely typed	Mandatory `.proto` file; strongly typed generated code
Streaming	Request/response; push needs SSE or WebSockets	Built-in server, client and bidirectional streaming
Transport	HTTP/1.1 or HTTP/2	HTTP/2 only
Browser support	Native: `fetch()`, curl, Postman	Needs gRPC-Web or a gateway

Why REST outside and gRPC inside

External clients such as browsers, mobile apps and third-party developers need an interface that is universal and easy to debug. REST with JSON works from any browser or command-line tool without generated code, and browsers cannot call raw gRPC. Internal services are all controlled by the same team, exchange many messages, and benefit from performance and strict contracts. gRPC gives smaller messages, lower latency, compile-time checking of the interface and native streaming. Part 3 follows exactly this split: clients use REST and SSE, while the coordinator and workers use gRPC for registration, heartbeats and streamed search progress.

1.4 Long-Running REST Operations and SSE

A proof-of-work search can run for minutes. If `POST /jobs` waited for the answer, the request would be cut off by client, proxy or load-balancer timeouts (often 30–60 s), and the client could neither see progress nor cancel. The **asynchronous job pattern** separates submission from execution:

1. **Submit:** the client sends `POST /jobs`. The server validates the input, starts the work in the background and immediately returns **202 Accepted** with a `job_id`.
2. **Track:** the client checks the job using its ID, either by **polling** `GET /jobs/{id}` until the status is terminal, or by **streaming** updates from `GET /jobs/{id}/events`.
3. **Control:** because the job is a resource, it can be listed (`GET /jobs`) and cancelled (`DELETE /jobs/{id}`).

The client can disconnect and come back later, several clients can watch the same job, and the server can run many jobs concurrently.



Figure 4. Asynchronous REST job submission followed by status polling.

Server-Sent Events (SSE) remove the need for repeated polling. The client opens an ordinary GET request; the server responds with `Content-Type: text/event-stream`, keeps the connection open and writes an event whenever there is news. Each event is a few text lines ending with a blank line:

SSE WIRE FORMAT

```
event: progress
data: {"status":"RUNNING","nonces_checked":170000000,"elapsed_seconds":8.43}

event: found
data: {"status":"FOUND", ...}
```

In a browser, the `EventSource` API consumes this stream and reconnects automatically; on the command line, `curl -N` prints events as they arrive. SSE is one-directional (server to client) and works over plain HTTP through proxies, which makes it simpler than WebSockets when the client only needs to listen. Actions such as cancellation still use separate HTTP requests.

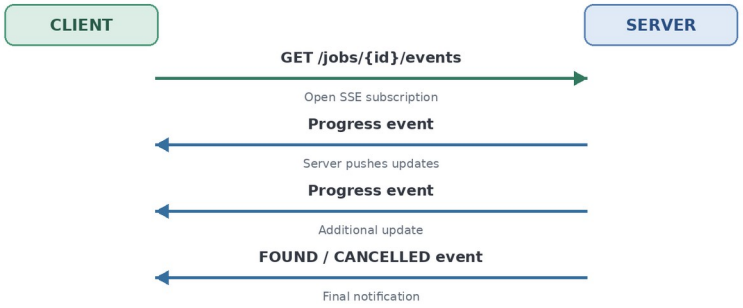


Figure 5. SSE progress events delivered over one persistent HTTP response.

2. A Streaming RPC Service in Go

2.1 Project Overview

A small **Live Log Viewer** was implemented in Go to demonstrate both a unary and a server-streaming RPC. The gRPC server listens on `localhost:50051`. The client first calls `GetServerInfo` (unary), then subscribes with `WatchLogs` (server streaming) and receives five log messages about one second apart. The log content is simulated for the demonstration.

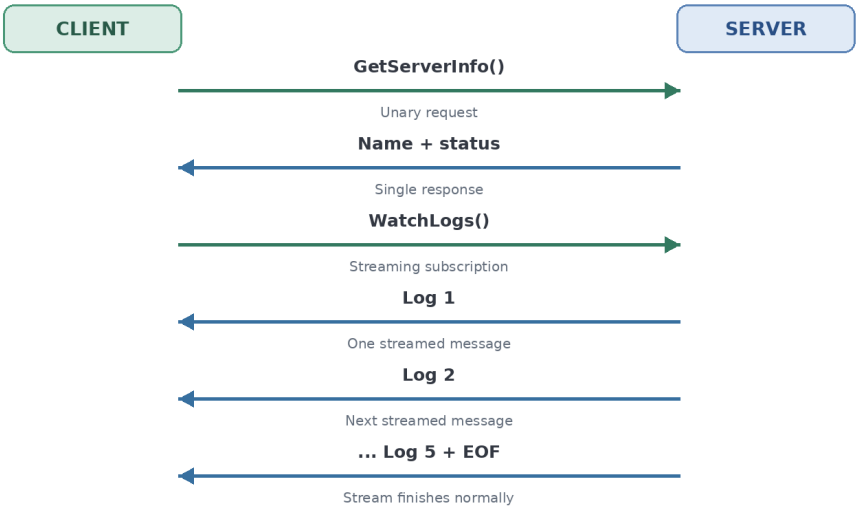


Figure 6. Live Log Viewer: gRPC method calls and message flow.

2.2 Protocol Definition

`proto/log.proto` defines the `LogService` contract and its messages. `GetServerInfo` returns one `InfoResponse`, whereas `WatchLogs` returns a stream of `LogMessage` objects.

PROTO/LOG.PROTO

```
syntax = "proto3";

package logs;
option go_package = "logviewer/proto";

service LogService {
  rpc GetServerInfo(InfoRequest) returns (InfoResponse);     // unary
  rpc WatchLogs(LogRequest)      returns (stream LogMessage); // server streaming
}

message InfoRequest  {}
message InfoResponse { string name = 1;  string status = 2; }
message LogRequest   { string service_name = 1; }
message LogMessage   { string level = 1; string message = 2; }
```

2.3 Implementation Summary

The server's `GetServerInfo` returns the name "Log Server" and status "RUNNING". `WatchLogs` sends five simulated log entries and checks the stream context before each send, so it stops as soon as the client

cancels. The client prints each message as soon as it arrives and treats `io.EOF` as the normal end of the stream.

Table 3. Part 2 project structure.

File	Responsibility
`proto/log.proto`	Service contract and message definitions
`server/main.go`	gRPC handlers and the simulated log stream
`client/main.go`	Unary request and stream receiver
`server/main_test.go`	Integration tests: server info, stream timing, cancellation, deadlines
`README.md`	Setup, execution and testing instructions

BUILD AND RUN (PART 2)

```
cd "Part 2/logviewer"
go build ./...
go run ./server              # terminal 1: starts gRPC server on :50051
go run ./client              # terminal 2: unary call, then streamed logs
go test -count=1 -v ./...    # integration and cancellation tests
```

2.4 Execution and Test Results

The client received the unary server information and all five streamed log messages, and the stream completed normally. In a second run the client was interrupted with Ctrl+C after two messages, and the server logged that the stream context was cancelled. The automated tests covering server information, stream timing, cancellation and deadlines all passed (`ok logviewer/server 4.368s`).

log.proto

logviewer > proto > log.proto

```
1  syntax = "proto3";
2
3  package logs;
4
5  option go_package = "logviewer/proto";
6
7  service LogService {
8    rpc GetServerInfo(InfoRequest) returns (InfoResponse);
9    rpc WatchLogs(LogRequest) returns (stream LogMessage);
10 }
11
12 message InfoRequest {}
13
14 message InfoResponse {
15   string name = 1;
16   string status = 2;
17 }
18
19 message LogRequest {
20   string service_name = 1;
21 }
22
23 message LogMessage {
24   string level = 1;
25   string message = 2;
26 }
```

Figure 7. The protocol definition with one unary and one server-streaming method.

```
===== Unary RPC =====
Server Name: Log Server
Status: RUNNING

===== Server Streaming RPC =====
Watching logs for demo-app...

[INFO] Server started
[INFO] User connected
[WARNING] High memory usage
[ERROR] Database connection failed
[INFO] Connection restored

Stream completed successfully.
```

Figure 8. Normal run: unary reply, then five streamed log messages.

```
===== Unary RPC =====
Server Name: Log Server
Status: RUNNING

===== Server Streaming RPC =====
Watching logs for demo-app...

[INFO] Server started
[INFO] User connected

Stream canceled.
```

Figure 9. Client cancelled with Ctrl+C after two messages.

```
2026/11/11 19:59:49 Log Server listening on localhost:50051
2026/11/11 20:00:37 GetServerInfo: client 127.0.0.1:65201 connected
2026/11/11 20:00:37 WatchLogs: watching service "demo-app"
2026/11/11 20:00:37 WatchLogs [demo-app]: [INFO] Server started
2026/11/11 20:00:38 WatchLogs [demo-app]: [INFO] User connected
2026/11/11 20:00:39 WatchLogs [demo-app]: [WARNING] High memory usage
2026/11/11 20:00:40 WatchLogs [demo-app]: [ERROR] Database connection failed
2026/11/11 20:00:41 WatchLogs [demo-app]: [INFO] Connection restored
2026/11/11 20:00:41 WatchLogs: service "demo-app" stream completed
2026/11/11 20:01:41 GetServerInfo: client 127.0.0.1:65207 connected
2026/11/11 20:01:41 WatchLogs: watching service "demo-app"
2026/11/11 20:01:41 WatchLogs [demo-app]: [INFO] Server started
2026/11/11 20:01:42 WatchLogs [demo-app]: [INFO] User connected
2026/11/11 20:01:43 WatchLogs: service "demo-app" stopped: context canceled
```

Figure 10. Server log: a completed stream followed by a cancelled stream.

```
=== RUN   TestServerInfo
2026/11/11 20:03:33 GetServerInfo: client 127.0.0.1:65112 connected
--- PASS: TestServerInfo (0.03s)
=== RUN   TestWatchLogsArriveOverTimeAndFinish
2026/11/11 20:03:33 WatchLogs: watching service "payments"
2026/11/11 20:03:33 WatchLogs [payments]: [INFO] Server started
    main_test.go:89: received at 20:03:33.441: [INFO] Server started
2026/11/11 20:03:34 WatchLogs [payments]: [INFO] User connected
    main_test.go:89: received at 20:03:34.441: [INFO] User connected
2026/11/11 20:03:35 WatchLogs [payments]: [WARNING] High memory usage
    main_test.go:89: received at 20:03:35.442: [WARNING] High memory usage
2026/11/11 20:03:36 WatchLogs [payments]: [ERROR] Database connection failed
    main_test.go:89: received at 20:03:36.442: [ERROR] Database connection failed
2026/11/11 20:03:37 WatchLogs [payments]: [INFO] Connection restored
2026/11/11 20:03:37 WatchLogs: service "payments" stream completed
    main_test.go:89: received at 20:03:37.442: [INFO] Connection restored
--- PASS: TestWatchLogsArriveOverTimeAndFinish (4.00s)
=== RUN   TestWatchLogsCancellation
2026/11/11 20:03:37 WatchLogs: watching service "payments"
2026/11/11 20:03:37 WatchLogs [payments]: [INFO] Server started
2026/11/11 20:03:37 WatchLogs: service "payments" stopped: context canceled
--- PASS: TestWatchLogsCancellation (0.00s)
=== RUN   TestWatchLogsDeadline
2026/11/11 20:03:37 WatchLogs: watching service "demo-app"
2026/11/11 20:03:37 WatchLogs [demo-app]: [INFO] Server started
2026/11/11 20:03:37 WatchLogs: service "demo-app" stopped: context canceled
--- PASS: TestWatchLogsDeadline (0.20s)
PASS
ok      logviewer/server        4.368s
```

Figure 11. Automated integration tests pass, including stream timing and cancellation.

3. Distributed Proof-of-Work Nonce Search

3.1 Objective and Proof-of-Work Rule

Given a string `data` and a difficulty `d`, the system must find a non-negative integer `nonce` such that the hexadecimal SHA-256 hash of `data` concatenated with the decimal form of `nonce` begins with `d` zeros. Difficulty is accepted in the range 1–8. Each additional zero makes a valid hash 16 times rarer, so the expected work grows as 16^d (about 1 million hashes for d = 5 and 4.3 billion for d = 8).

For example, for `data = "hello"` and `d = 5`, nonce 1,057,335 is valid because SHA256("hello1057335") = `00000504ae0d`.... The solution returned is not necessarily the smallest valid nonce, because workers search different ranges in parallel and the first one to succeed wins. The coordinator re-computes the hash of every proposed solution before accepting it.

3.2 System Architecture

The system consists of one Go **coordinator** process and several independently launched Go **worker** processes:

- The coordinator's **HTTP server** serves the REST API, the SSE event streams and the optional browser dashboard.
- The coordinator's **gRPC server** receives `RegisterWorker` and `Heartbeat` calls from workers.
- The **job manager and scheduler** splits the nonce space into 1,000,000-nonce ranges, gives every range assignment its own cancellable context, keeps a retry queue for failed ranges, and verifies results.
- Each **worker** hosts its own gRPC `Search` service and searches one assigned range at a time, streaming progress back to the coordinator.

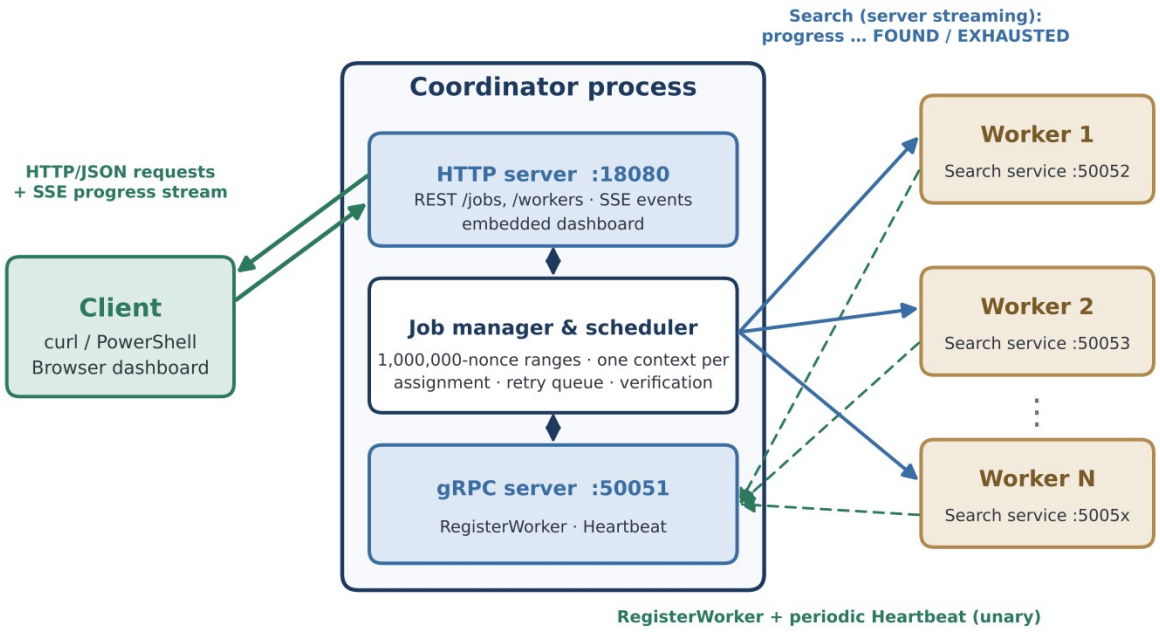


Figure 12. System architecture: REST/SSE client, coordinator components and independent gRPC workers.

3.3 REST API and Job Lifecycle

Table 4. Coordinator REST endpoints.

Method	Endpoint	Behaviour
POST	`/jobs`	Validate `{"data", "difficulty"}`; return **202 Accepted** and `{"job_id"}`
GET	`/jobs/{id}`	Status (RUNNING / FOUND / CANCELLED), nonces checked, elapsed time, and nonce and hash once found
GET	`/jobs/{id}/events`	Live progress and a final event over SSE
DELETE	`/jobs/{id}`	Cancel a running job and stop all of its worker searches
GET	`/jobs`	List all jobs and their statuses
GET	`/workers`	List registered workers, liveness and current assignment

Each job starts as RUNNING and moves to exactly one terminal state, FOUND or CANCELLED, which is never reversed. Errors are reported as JSON with the appropriate status code:

Table 5. REST error handling.

Situation	Status	Response body
Missing data or difficulty outside 1–8	400 Bad Request	`{"error":"nonempty data and integer difficulty 1..8 required"}`
Unknown job ID	404 Not Found	`{"error":"job not found"}`
Cancel a job that already finished	409 Conflict	`{"error":"job already terminal"}`

Elapsed time is measured from submission. The nonce count is the sum of progress reported by workers, so work lost in a crashed worker is not counted.

3.4 gRPC Services and Worker Registration

Table 6. gRPC methods between coordinator and workers.

RPC	Direction	Type	Purpose
`RegisterWorker`	Worker → Coordinator	Unary	Register a unique worker ID and its Search address
`Heartbeat`	Worker → Coordinator	Unary	Refresh the worker's liveness lease
`Search`	Coordinator → Worker	Server streaming	Search one range; stream progress, then FOUND or EXHAUSTED

A worker starts its gRPC listener first, then registers with the coordinator and receives a session ID, then sends a heartbeat every 500 ms. A worker that misses heartbeats for 3 s is marked dead. Each `Search` request carries the job ID, data, difficulty, an assignment ID and a half-open range `[start, end)`. The worker streams progress messages with the number of nonces checked, then a final message with either the valid nonce and hash (FOUND) or a statement that its range contains no solution (EXHAUSTED). The coordinator ignores results from stale assignments and verifies every proposed nonce.

PROTO/POW.PROTO

```
syntax = "proto3";
package pow;
option go_package = "pow-distributed/proto;pb";

service Coordinator {
  rpc RegisterWorker(RegisterRequest) returns (RegisterResponse);
  rpc Heartbeat(HeartbeatRequest)      returns (HeartbeatResponse);
}
service Worker {
  rpc Search(SearchRequest) returns (stream SearchUpdate);
}

message RegisterRequest {
  string worker_id = 1;  string address = 2;  string instance_id = 3;
}
message RegisterResponse { string session_id = 1; }
message HeartbeatRequest { string worker_id = 1; string session_id = 2; }
message HeartbeatResponse {}

message SearchRequest {
  string job_id = 1;  string data = 2;  uint32 difficulty = 3;
  uint64 start = 4;   uint64 end = 5;   string assignment_id = 6;
}
enum UpdateKind { PROGRESS = 0; FOUND = 1; EXHAUSTED = 2; }
message SearchUpdate {
  string assignment_id = 1;  uint64 checked = 2;  UpdateKind kind = 3;
  uint64 nonce = 4;          string hash = 5;
}
```

3.5 Range Scheduling and Concurrent Jobs

The coordinator hands out non-overlapping ranges of 1,000,000 nonces: `[0, 1000000)`, `[1000000, 2000000)` and so on. Each idle worker receives the next unassigned range through a `Search` call. When a worker reports EXHAUSTED, it is immediately assigned the next range. Several jobs can run at the same time; the scheduler shares workers between them fairly at range boundaries. Each worker is an independent operating-system process, and goroutines handle concurrent RPCs inside each process.

3.6 Early Termination, Cancellation and Fault Tolerance

How cancellation travels from HTTP down to the workers

Every `Search` RPC is started with its own cancellable Go `context`, one per range assignment (with a 2-minute safety timeout). The coordinator records which worker is running which assignment for which job, so ending a job cancels exactly the RPCs that belong to it:

1. The client sends `DELETE /jobs/{id}` (or a worker reports a valid nonce, for early termination).
2. The coordinator marks the job CANCELLED (or FOUND). For every worker whose active assignment belongs to this job, it calls that assignment's cancel function and logs `cancel search`.
3. Cancelling an assignment's context cancels its `Search` call, and gRPC propagates the cancellation over HTTP/2 to the worker.
4. On each worker, the stream's context is cancelled. The search loop checks `ctx.Done()` regularly, stops immediately and logs `search cancelled … reason="context canceled"`.

5. The REST response returns the CANCELLED job, and every SSE subscriber receives a final event before its stream closes.

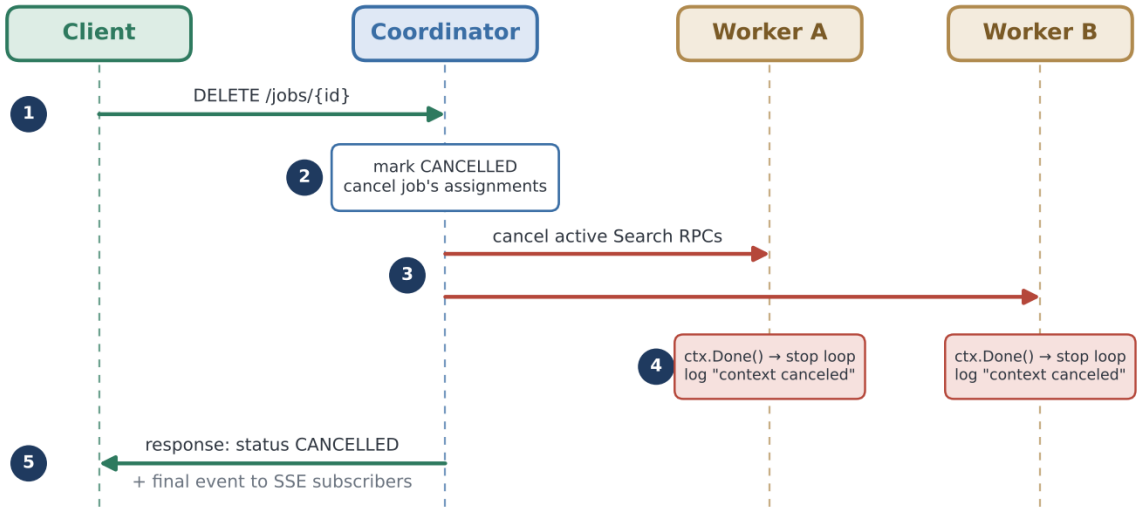


Figure 13. Cancellation path from DELETE /jobs/{id} to every worker searching for the job.

Early termination uses the same mechanism. As soon as one worker returns a valid nonce, the coordinator verifies it, marks the job FOUND and cancels the remaining assignments of that job, which stops every other worker still searching for it.

Fault tolerance

If a worker misses heartbeats for longer than the 3-second timeout or a `Search` call fails, the coordinator marks the worker unavailable and puts its unfinished range back in the retry queue. The range is then reassigned to a healthy worker under a new assignment ID. Assignment IDs and per-session tokens ensure that a late result from an abandoned session cannot corrupt the job. A range may be searched twice during a retry, but no range is ever skipped.

3.7 Configuration and Running the System

The project is in `Part 3/pow-distributed`. All required settings are command-line flags of the Go programs. Helper PowerShell scripts wrap these flags, start the processes in the background, assign unique worker ports and write logs to the `.run` directory.

Table 7. Configurable parameters.

Setting	Go flag	Script parameter	Value used in tests
Coordinator HTTP address	coordinator `-http`	`-HttpPort`	127.0.0.1:18080 (8080 was in use)
Coordinator gRPC address	coordinator `-grpc`	`-GrpcPort`	127.0.0.1:50051
Range (chunk) size	coordinator `-chunk-size`	`-ChunkSize`	1,000,000 nonces
Heartbeat timeout	coordinator `-heartbeat-timeout`	`-HeartbeatTimeout`	3 s (default)
Number of workers	one process per worker	`-Count`	1, 2, 4 or 8
Worker gRPC port	worker `-port`	`-BasePort` (+1 per worker)	50052, 50053, …
Heartbeat interval	worker `-heartbeat-interval`	`-HeartbeatInterval`	500 ms (default)

BUILD AND RUN (WINDOWS POWERSHELL)

```
cd "Part 3/pow-distributed"
go build ./...
.\scripts\start-coordinator.ps1 -HttpPort 18080 -ChunkSize 1000000
.\scripts\start-workers.ps1 -Count 4
Start-Process "http://localhost:18080/"     # optional dashboard

# equivalent manual start (one terminal per process)
go run ./cmd/coordinator -http 127.0.0.1:18080 -grpc 127.0.0.1:50051 `
                         -chunk-size 1000000
go run ./cmd/worker -id worker-1 -port 50052 -coordinator 127.0.0.1:50051

# client example
curl.exe -X POST http://localhost:18080/jobs -H "Content-Type: application/json" `
         -d '{"data":"hello","difficulty":5}'
curl.exe -N http://localhost:18080/jobs/<job_id>/events

.\scripts\stop-system.ps1
```

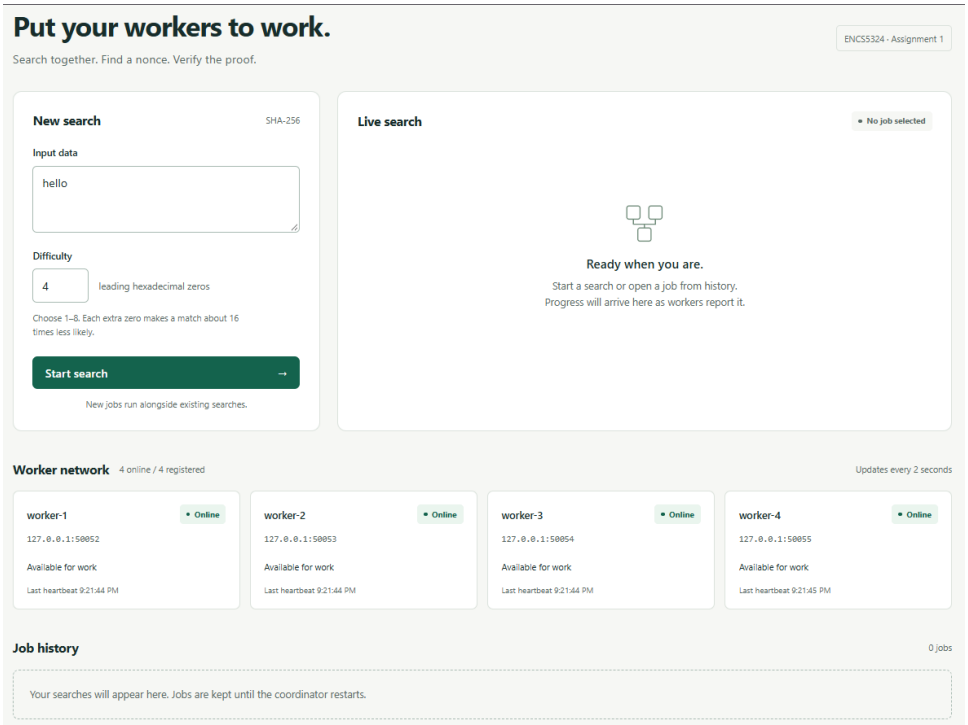


Figure 14. Dashboard showing four registered, online worker processes.

3.8 Optional Browser Dashboard (Bonus)

The coordinator also serves a single-page dashboard built with embedded HTML, CSS and JavaScript, so no separate frontend server or build step is needed. It submits jobs, shows live progress through the browser's native `EventSource` API, cancels jobs, shows worker health and assignments, and lists job history.

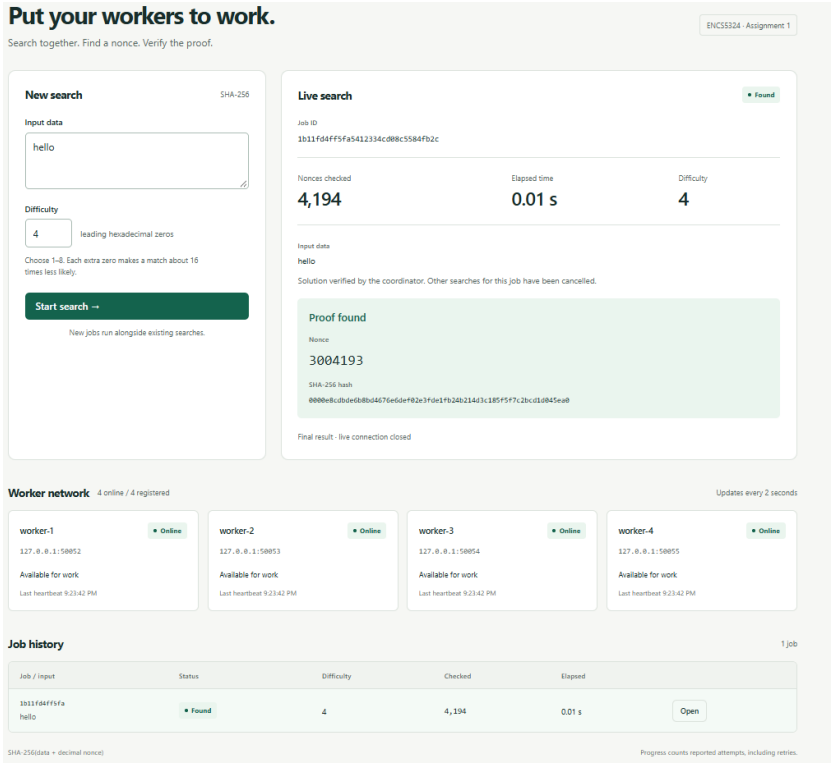


Figure 15. A completed difficulty-4 search displayed in the dashboard.

4. Testing and Experiments

All tests ran on one Windows PC with the coordinator on HTTP port 18080 and gRPC port 50051, using four worker processes unless stated otherwise.

4.1 REST Workflow and SSE

Jobs were submitted with `POST /jobs`, their status was read with `GET /jobs/{id}`, and progress was watched with `curl.exe -N` on `GET /jobs/{id}/events`. The stream delivered a series of RUNNING progress events and, when the search finished, a final `found` event. A client that subscribed after the job had finished immediately received the final snapshot.

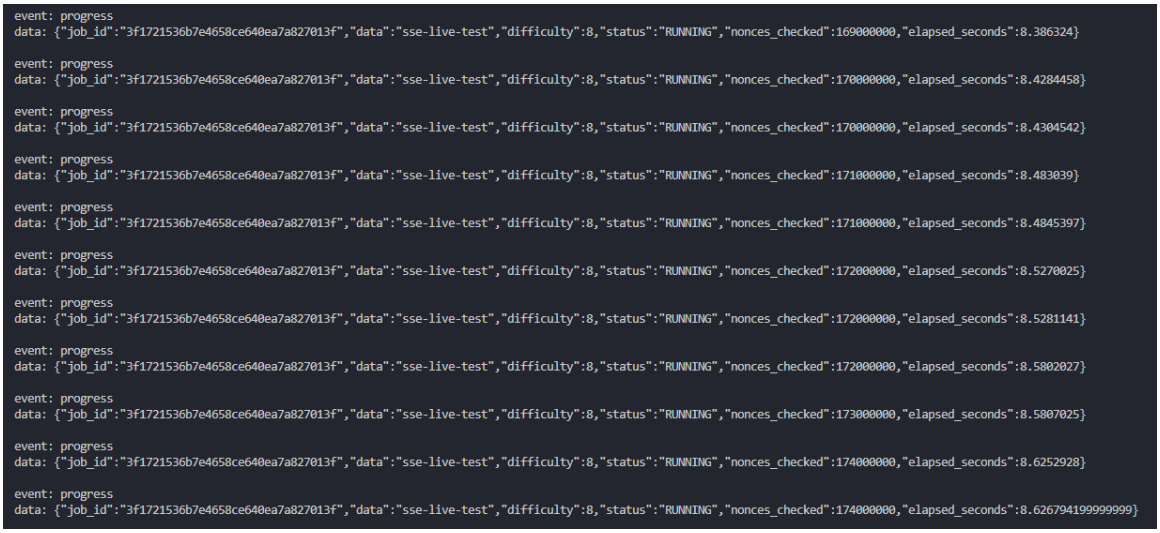


Figure 16. Live SSE progress events received in PowerShell.

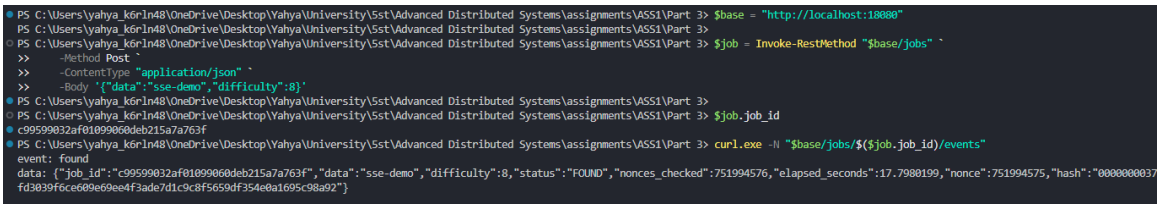


Figure 17. A late SSE subscriber immediately receives the final FOUND event.

4.2 Difficulty Experiment (d = 4, 5, 6)

The string "hello" was searched at difficulties 4, 5 and 6 as required, plus difficulty 8 as an additional stress test. Times are the coordinator's `elapsed_seconds` values from the job list (Section 4.4). All four nonces were re-hashed independently (Appendix B) and are valid.

Table 8. Difficulty results with four workers (* additional stress test).

d	Nonce	SHA-256 of "hello" + nonce	Checked	Time
4	3,004,193	`0000e8cdbde6b8bd4676e6def02e3fde1fb24b214d3c185f5f7c2bcd1d045ea0`	4,194	0.006 s
5	1,057,335	`00000504ae0d531793668fe57e13bb540a80aec130ca1147db532d38b799e1fd`	57,336	0.011 s
6	33,290,382	`000000865194de5c0744d72183dcce2eca2b69264b749a1798f1b6996baef7de`	32,290,383	1.04 s
8*	1,202,655,595	`00000000e0d103722c6e172e145d15b9e909e109b47b30d741911bdb3cf3da84`	1,200,655,596	38.23 s

Each extra zero multiplies the expected work by 16, and the measured time grows accordingly from milliseconds (d = 4, 5) to about one second (d = 6) and 38 seconds (d = 8). Individual times vary a lot because the position of the first valid nonce is random. At d = 6 and d = 8 the system sustained about **31 million hashes per second** with four workers (32.3 M / 1.04 s and 1,200.7 M / 38.23 s), so the throughput is stable once jobs are long enough.

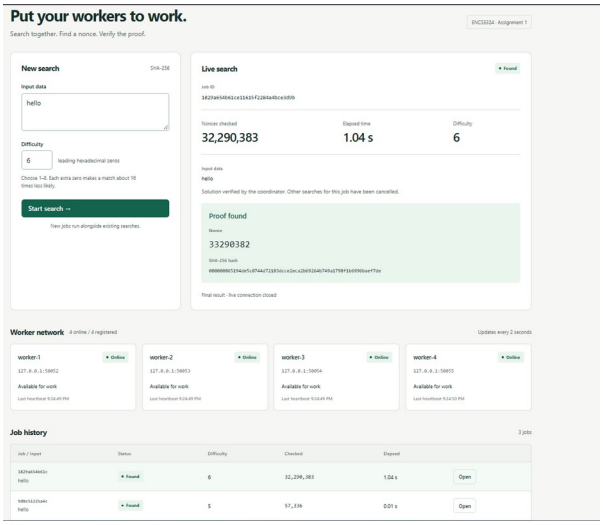


Figure 18. Difficulty 6: nonce, verified proof and duration.

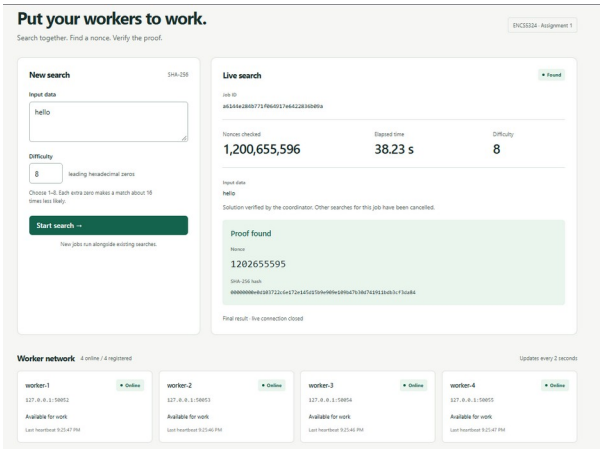


Figure 19. Difficulty 8: a much longer workload.

4.3 Scalability: 1, 2, 4 and 8 Workers

The d = 5 test on "hello" was repeated with 1, 2, 4 and 8 worker processes, with a range size of 1,000,000. For each configuration a fresh coordinator was started and five jobs were run in sequence. The median of the five `elapsed_seconds` values is reported because the times are only a few milliseconds and sensitive to outliers.

Table 9. Raw scalability measurements (seconds), d = 5.

Workers	Run 1	Run 2	Run 3	Run 4	Run 5	Median (s)
1	0.0320925	0.0226070	0.0219993	0.0210195	0.0220462	**0.0220462**
2	0.0069988	0.0066388	0.0070026	0.0070037	0.0069946	**0.0069988**
4	0.0075312	0.0085045	0.0069992	0.0075016	0.0080034	**0.0075312**
8	0.0085022	0.0064647	0.0069967	0.0060031	0.0059995	**0.0064647**

Table 10. Summary: winning nonce, work done and speedup relative to one worker.

Workers	Nonce found	Winning range	Nonces checked	Median time	Speedup
1	156,056	[0, 1M)	156,057	22.05 ms	1.00×
2	1,057,335	[1M, 2M)	57,336	7.00 ms	3.15×
4	1,057,335	[1M, 2M)	57,336	7.53 ms	2.93×
8	6,028,023	[6M, 7M)	28,024	6.46 ms	3.41×

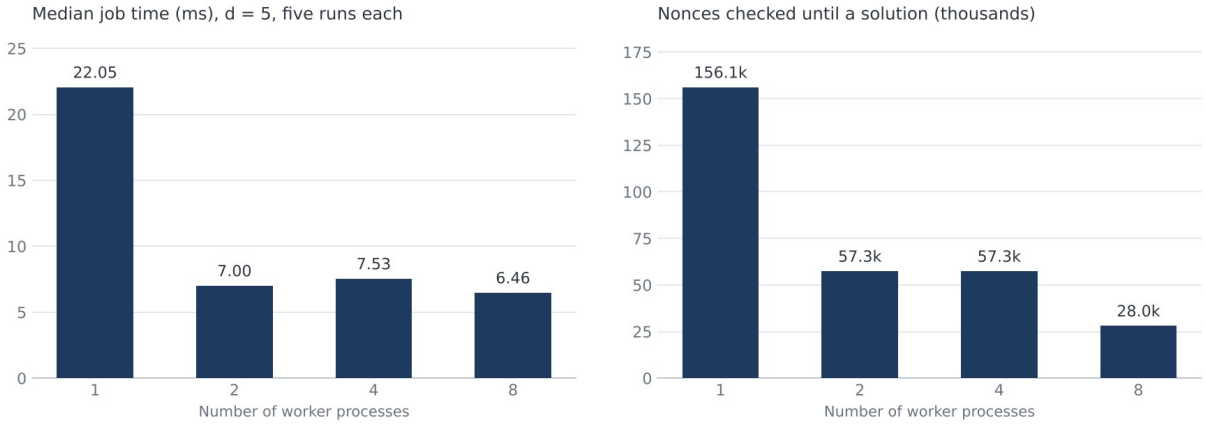


Figure 20. Median job time (left) and nonces checked before a solution was found (right) for each worker count.

Discussion

Adding workers reduced the median job time from 22.05 ms to between 6.5 and 7.5 ms, a speedup of about 3×. However, this is **not** the result of the workers hashing faster. The nonce columns show why:

- With **one worker**, the search runs through `[0, 1M)` in order and the first valid nonce there is 156,056, so 156,057 hashes are needed.
- With **two workers**, the second worker searches `[1M, 2M)` in parallel and finds 1,057,335 after only 57,336 hashes. This shorter search accounts for most of the 3.15× speedup (156,057 / 57,336 = 2.72×).
- With **four workers**, the extra ranges `[2M, 3M)` and `[3M, 4M)` have their first solutions later (at offsets 65,325 and 597,266), so the same nonce wins and the time stays the same (7.53 ms versus 7.00 ms is within run-to-run variation).
- With **eight workers**, range `[6M, 7M)` contains a solution after only 28,024 hashes. The work halves again, but the time improves only from 7.00 ms to 6.46 ms.

Key finding

At d = 5 a job finishes in a few milliseconds, so the result is dominated by fixed costs (RPC set-up, scheduling, progress reporting and context switching between up to nine processes on one PC) and by where the first valid nonce happens to fall. The speedup therefore reflects a lucky search order more than raw parallel throughput.

Longer jobs show the real benefit of parallelism: with four workers the system sustained about 31 million hashes per second at d = 6 and d = 8, compared with about 7 million per second for the single-worker d = 5 runs (156,057 hashes in 22 ms). A fairer scalability test would use a fixed amount of work, for example a higher difficulty or ranges that contain no solution.

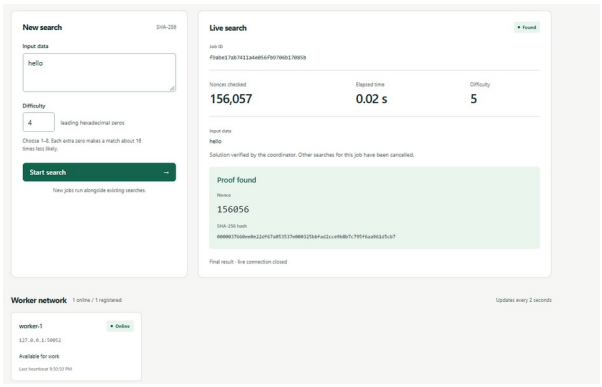


Figure 21. One worker: baseline difficulty-5 run.

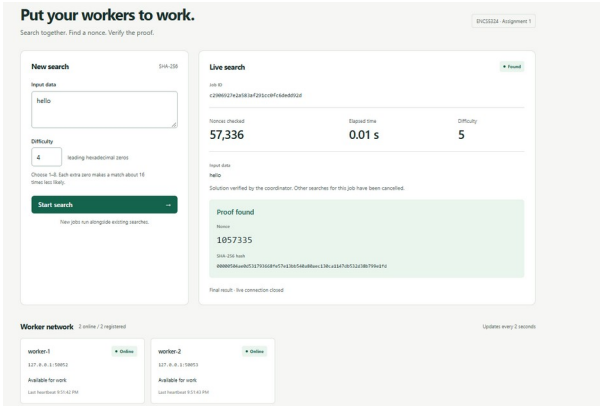


Figure 22. Two workers: successful difficulty-5 run.

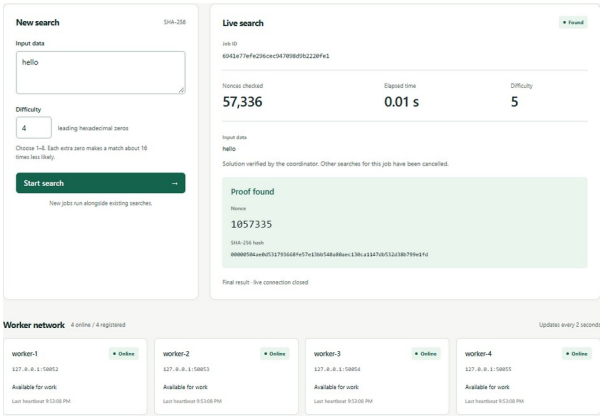


Figure 23. Four workers: successful difficulty-5 run.

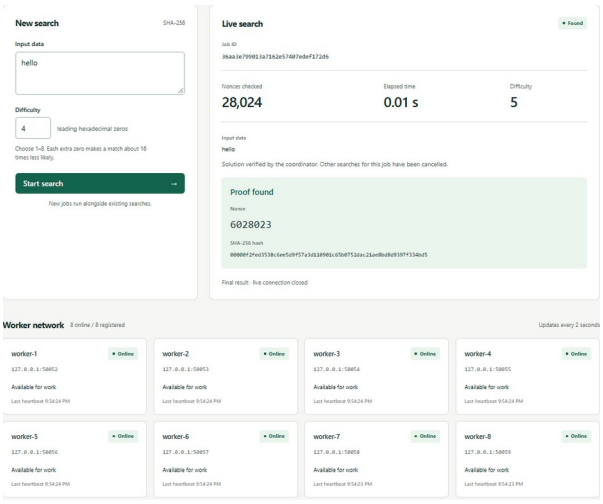


Figure 24. Eight workers: successful difficulty-5 run.

4.4 Concurrent Jobs

Three jobs with data "first", "second" and "third" at difficulty 6 were submitted in quick succession. Each received its own job ID, they ran at the same time on the shared workers, and all three reached FOUND with valid nonces (verified in Appendix B). The "second" job took 5.06 s, so the other two jobs ran entirely while it was still in progress.

Table 11. Three concurrent jobs.

Data	d	Nonce	Hash (prefix)	Checked	Elapsed	Status
first	6	100,794	`00000068069f1eaa…`	100,795	0.053 s	FOUND
second	6	46,052,729	`0000000638dad7ba…`	46,052,730	5.061 s	FOUND
third	6	7,427,171	`00000009e365ea49…`	7,427,172	0.962 s	FOUND

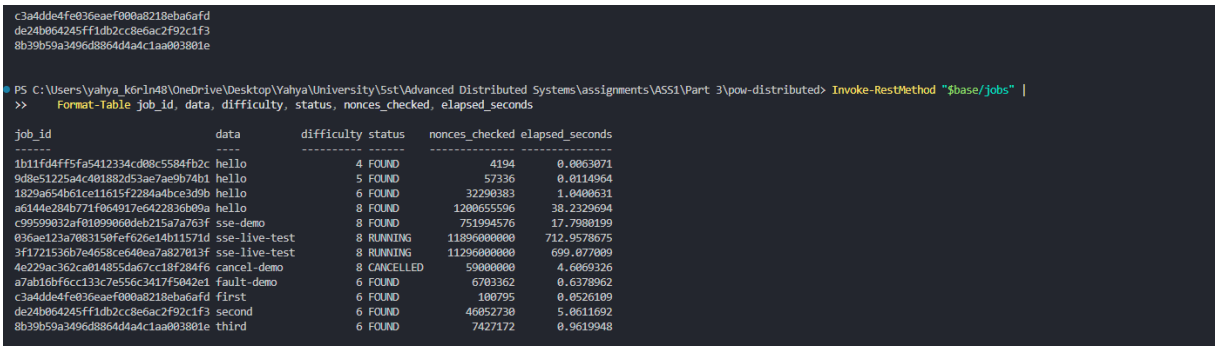


Figure 25. Job list after the concurrent test: first, second and third all FOUND.

4.5 Client-Initiated Cancellation

A difficulty-8 job with data "cancel-demo" was cancelled with `DELETE /jobs/{id}` while it was running. The response and a follow-up GET both reported CANCELLED after 59,000,000 nonces and 4.61 s. The coordinator log shows `client cancelled job` followed by `cancel search` for each active assignment, and the logs of worker-1 and worker-3, which were searching ranges for that job, show `search cancelled … reason="context canceled"` at the same second (21:30:52). Workers that had no active range for this job had nothing to cancel.



Figure 26. REST cancellation request and the persistent CANCELLED state.

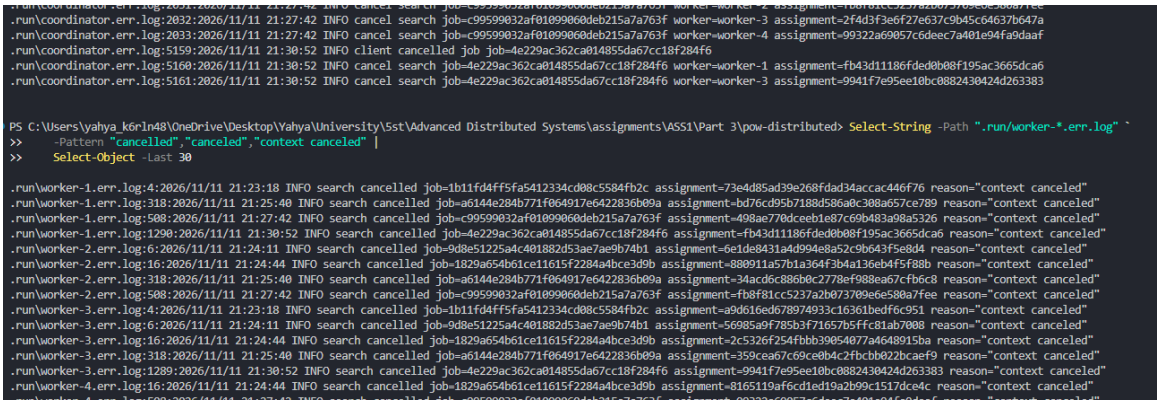


Figure 27. Coordinator and worker logs showing context cancellation reaching the workers.

4.6 Fault Tolerance and Range Reassignment

A difficulty-6 job ("fault-demo") was started and worker-2 was killed while it was searching the range `[2,000,000, 3,000,000)`. The coordinator's Search call failed with `code = Unavailable`, the coordinator logged `range requeued` for that exact range and assigned it to worker-3 under a new assignment ID. The job still completed with nonce 6,703,361 and hash `000000b6d5017402…` (verified in Appendix B), and the dashboard showed worker-2 as offline.

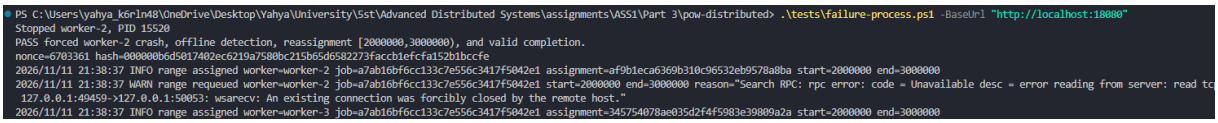


Figure 28. Worker-2 is stopped, its range is requeued and reassigned to worker-3.

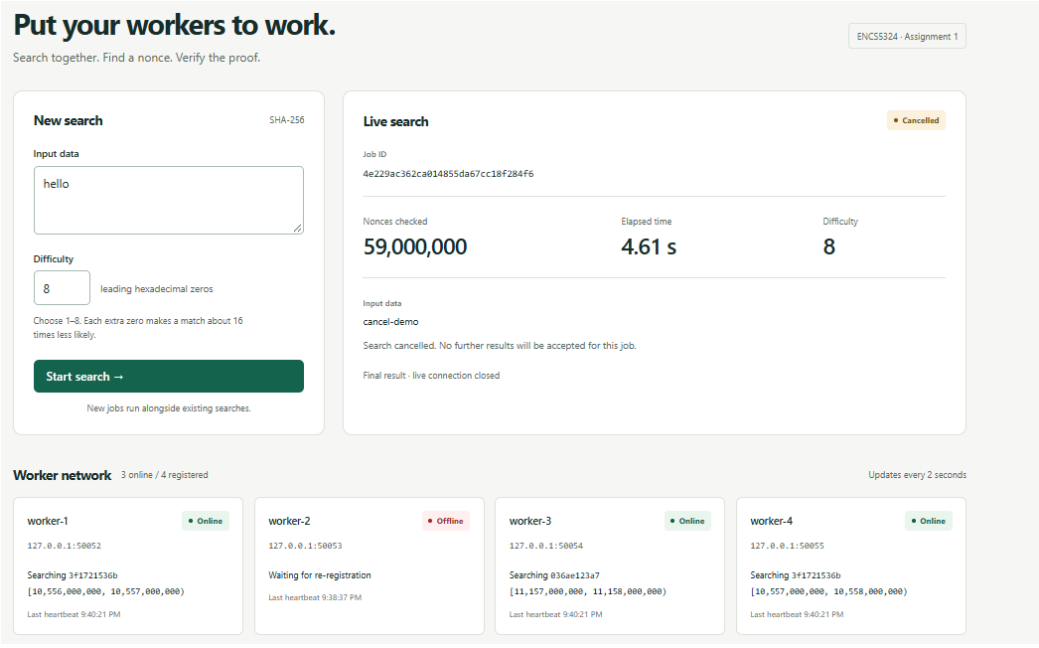


Figure 29. The dashboard reports worker-2 offline while the other workers remain active.

4.7 REST Error Handling

All three required error cases return the correct status code and a JSON error message (the response bodies are listed in Section 3.3). Invalid difficulty was tested twice: once with an inline body and once with a JSON file, to rule out PowerShell quoting problems.

Table 12. Required HTTP error responses.

Scenario	Expected	Observed
Difficulty 9	400 Bad Request	400 Bad Request
Unknown job ID	404 Not Found	404 Not Found
DELETE an already finished job	409 Conflict	409 Conflict

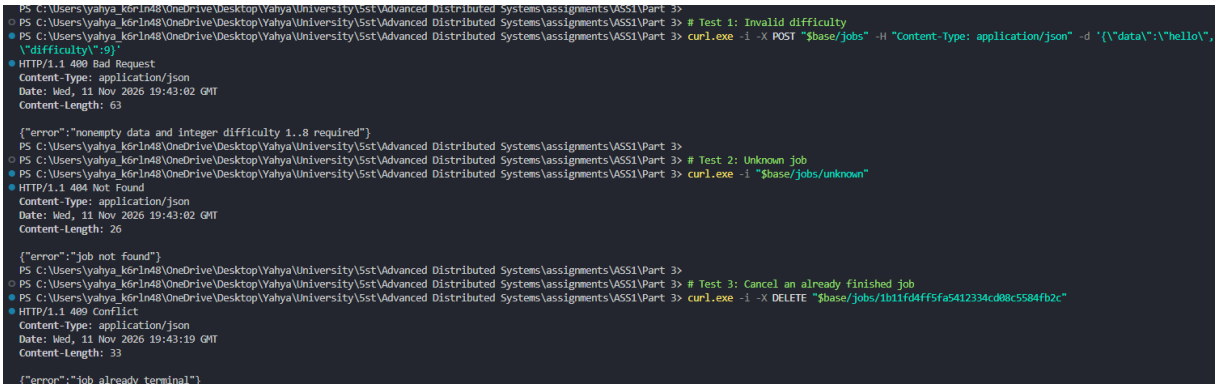


Figure 30. curl output for the 400, 404 and 409 cases.

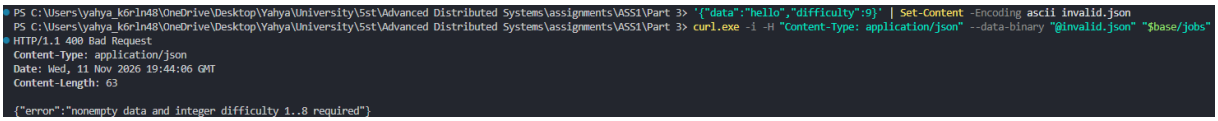


Figure 31. A valid JSON body with difficulty 9 also returns 400 Bad Request.

4.8 Automated Tests, Bonus and Limitations

According to the project's `TESTING.md`, `go build`, `go vet`, `go test` and `go test -race` all completed successfully, with 18 top-level tests and two subtests passing, together with process-level checks of failure and reassignment, the launch scripts and the dashboard. The optional dashboard (Section 3.8) was used throughout Part 4 to show live job states via `EventSource` and the status of each worker.

The system was built for a local educational setting and has these limitations:

- Jobs are kept in memory only, so a coordinator restart loses all job state.
- There is no authentication on the REST or gRPC interfaces.
- The coordinator is a single point of failure; only worker failures are tolerated.
- All processes shared one PC, so the scalability results include CPU contention between workers.

5. Conclusion

This assignment combined two styles of distributed communication. Part 1 explained synchronous and asynchronous RPC, the four gRPC method types, the trade-offs between REST and gRPC, and how the asynchronous job pattern with SSE handles long-running work. Part 2 implemented a gRPC Live Log Viewer in Go with a unary and a server-streaming method, verified by normal runs, client cancellation and automated tests.

Parts 3 and 4 implemented and evaluated a coordinator–worker proof-of-work system. REST and SSE serve clients; gRPC distributes ranges and streams progress; and Go context cancellation carries both early termination and client cancellation from a single HTTP request down to every worker. The tests showed correct results at every difficulty, three jobs completing concurrently, prompt cancellation, a failed worker's range being reassigned without restarting the job, and correct 400/404/409 responses. The scalability experiment showed a 3.4× faster median with eight workers, and its analysis showed that for very short jobs the gain comes mainly from parallel ranges finding an earlier solution, while longer jobs reach a stable throughput of about 31 million hashes per second with four workers.

Appendix A — Source Code, Build and Reproduction

GitHub repository: https://github.com/yahyasarhan22/Distributed-Proof-of-Work-Nonce-Search-with-REST-and-gRPC

The repository contains the complete source code for Parts 2 and 3, including both `.proto` files (listed in full in Sections 2.2 and 3.4), the generated Go bindings, the test suites and the benchmark CSV files.

Table 13. Source code layout.

Project	Contents
Part 2/logviewer/	`proto/log.proto`, generated Go bindings, `server/main.go`, `client/main.go`, `server/main_test.go`, README.md
Part 3/pow-distributed/	`proto/pow.proto`, `cmd/coordinator`, `cmd/worker`, `internal/coordinator` (REST, SSE, scheduler, dashboard), `internal/worker`, `internal/pow`, scripts in `scripts/` and `tests/`, benchmark CSVs, README.md, ARCHITECTURE.md, TESTING.md

BUILD, TEST AND RUN

```
# Part 2
cd "Part 2/logviewer"
go build ./...
go test -count=1 -v ./...
go run ./server        # terminal 1
go run ./client        # terminal 2

# Part 3
cd "Part 3/pow-distributed"
go build ./...
go test -count=1 -v ./...
.\scripts\start-coordinator.ps1 -HttpPort 18080 -ChunkSize 1000000
.\scripts\start-workers.ps1 -Count 4
.\tests\failure-process.ps1 -BaseUrl "http://localhost:18080"
.\scripts\stop-system.ps1
```

README.md, ARCHITECTURE.md and TESTING.md in the Part 3 project contain the full configuration options and verification steps.

Appendix B — Independent Hash Verification

Every nonce reported in Part 4 was checked outside the system with the short script below, which recomputes SHA-256 of the data followed by the decimal nonce and checks the number of leading zeros. All eight results are valid.

VERIFY.PY

```
import hashlib

def verify(data, nonce, d):
    h = hashlib.sha256(f"{data}{nonce}".encode()).hexdigest()
    return h, h.startswith("0" * d)

print(verify("hello", 1057335, 5))   # ('00000504ae0d...', True)
```

Table 14. Independently recomputed hashes for every reported solution.

Data	d	Nonce	SHA-256(data + nonce)
hello	4	3004193	0000e8cdbde6b8bd4676e6def02e3fde1fb24b214d3c185f5f7c2bcd1d045ea0
hello	5	1057335	00000504ae0d531793668fe57e13bb540a80aec130ca1147db532d38b799e1fd
hello	6	33290382	000000865194de5c0744d72183dcce2eca2b69264b749a1798f1b6996baef7de
hello	8	1202655595	00000000e0d103722c6e172e145d15b9e909e109b47b30d741911bdb3cf3da84
first	6	100794	00000068069f1eaabdff82238c9a0046e3d9fdb158a2800f777280b0d94c47cf
second	6	46052729	0000000638dad7ba5ea7ed1f2e59ba326313c46c37eb5f8e9fd0f1aa3a150588
third	6	7427171	00000009e365ea49ff175b8757daf2bc358ea06a50395450c4714c17bd697d18
fault-demo	6	6703361	000000b6d5017402ec6219a7580bc215b65d6582273faccb1efcfa152b1bccfe

Appendix C — References

[1] ENCS5324 Assignment #1: Distributed Proof-of-Work Nonce Search with REST and gRPC. Advanced Distributed Systems, Birzeit University, 2026.

[2] gRPC Documentation — Core concepts, architecture and lifecycle. https://grpc.io/docs/what-is-grpc/core-concepts/

[3] Protocol Buffers Documentation (proto3 language guide). https://protobuf.dev/programming-guides/proto3/

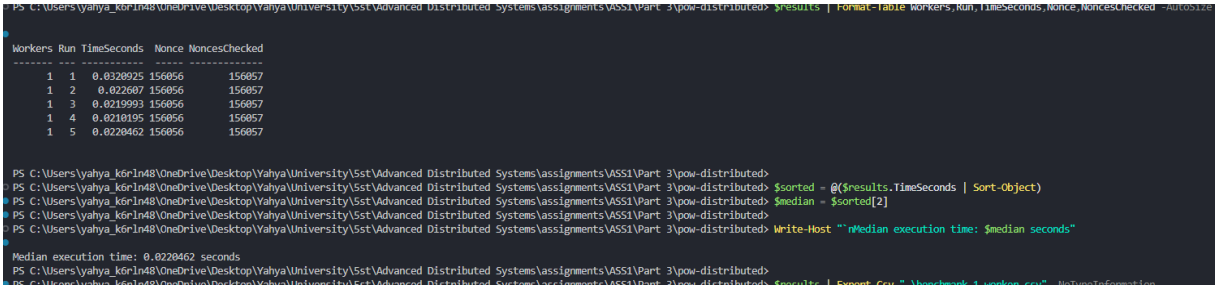
[4] The Go Programming Language Documentation; package context. https://pkg.go.dev/context

[5] MDN Web Docs — Using server-sent events. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

[6] Project source code and documentation (README.md, ARCHITECTURE.md, TESTING.md). https://github.com/yahyasarhan22/Distributed-Proof-of-Work-Nonce-Search-with-REST-and-gRPC

Appendix D — Benchmark Screenshots

Raw output of the five-run benchmark for each worker count, including the nonce found and the nonces checked.

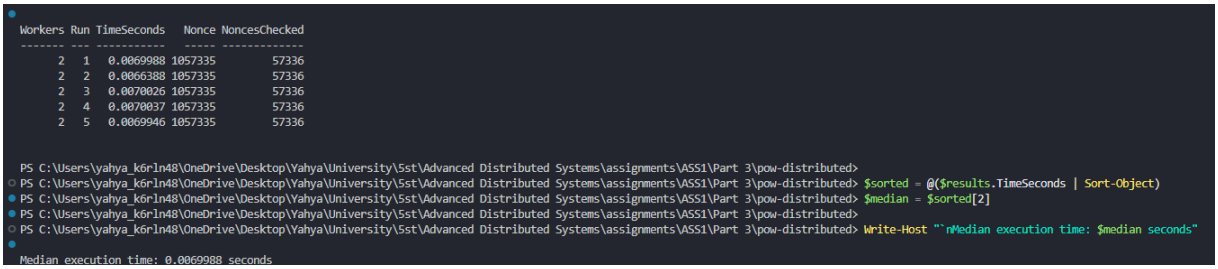


```
Workers Run TimeSeconds  Nonce NoncesChecked
------- --- -----------  ----- -------------
      1   1  0.0320025 156056        156057
      1   2   0.022607 156056        156057
      1   3  0.0219993 156056        156057
      1   4  0.0210195 156056        156057
      1   5  0.0220462 156056        156057


PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed>
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> $sorted = @($results.TimeSeconds | Sort-Object)
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> $median = $sorted[2]
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed>
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> Write-Host "`nMedian execution time: $median seconds"

Median execution time: 0.0220462 seconds
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed>
```

Figure 32. One worker: five runs and median.

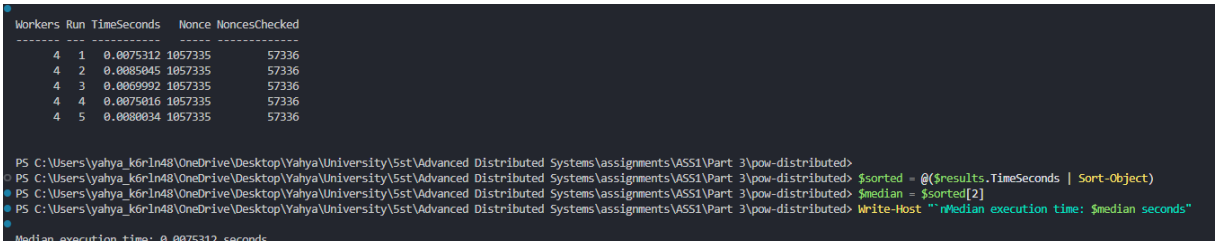


```
Workers Run TimeSeconds   Nonce NoncesChecked
------- --- -----------   ----- -------------
      2   1   0.0069988 1057335         57336
      2   2   0.0066388 1057335         57336
      2   3   0.0070026 1057335         57336
      2   4   0.0070037 1057335         57336
      2   5   0.0069946 1057335         57336


PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed>
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> $sorted = @($results.TimeSeconds | Sort-Object)
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> $median = $sorted[2]
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed>
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> Write-Host "`nMedian execution time: $median seconds"

Median execution time: 0.0069988 seconds
```

Figure 33. Two workers: five runs and median.

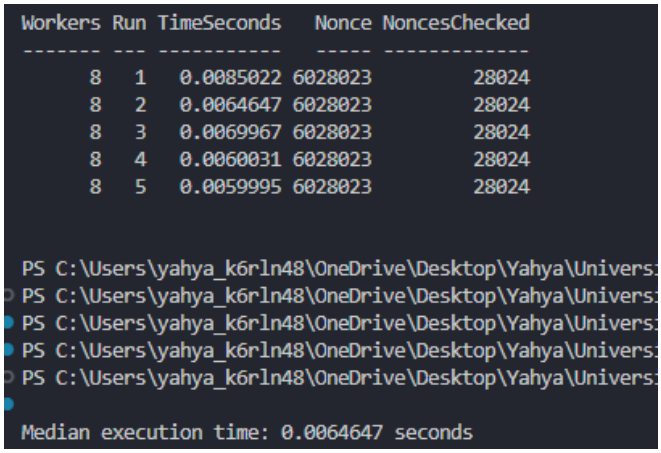


```
Workers Run TimeSeconds   Nonce NoncesChecked
------- --- -----------   ----- -------------
      4   1   0.0075312 1057335         57336
      4   2   0.0085045 1057335         57336
      4   3   0.0069992 1057335         57336
      4   4   0.0075016 1057335         57336
      4   5   0.0080034 1057335         57336


PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed>
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> $sorted = @($results.TimeSeconds | Sort-Object)
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> $median = $sorted[2]
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\University\5st\Advanced Distributed Systems\assignments\ASS1\Part 3\pow-distributed> Write-Host "`nMedian execution time: $median seconds"

Median execution time: 0.0075312 seconds
```

Figure 34. Four workers: five runs and median.



```
Workers Run TimeSeconds   Nonce NoncesChecked
------- --- -----------   ----- -------------
      8   1   0.0085022 6028023         28024
      8   2   0.0064647 6028023         28024
      8   3   0.0069967 6028023         28024
      8   4   0.0060031 6028023         28024
      8   5   0.0059995 6028023         28024


PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\Univers
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\Univers
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\Univers
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\Univers
PS C:\Users\yahya_k6rln48\OneDrive\Desktop\Yahya\Univers

Median execution time: 0.0064647 seconds
```

Figure 35. Eight workers: five runs and median.